MPCache: Model-Predictive Caching for Training-Free Video Diffusion Acceleration

Anonymous Authors

Abstract

Training-free caching accelerates a video diffusion model by evaluating a subset of the denoising steps and reusing the last exact output elsewhere. Existing methods decide which steps to evaluate with a threshold on a quantity that has already been observed, and spend their design effort on how a skipped step is reconstructed. We measure what a skipped step actually costs, and the cost turns out to be positional. On CogVideoX-2B at thirty steps, skipping the first step costs 0.57 LPIPS against the full run while every step after the tenth costs about 0.04, so the first ten steps carry about three quarters of the sensitivity, and two prompts agree on the order at 0.94 correlation. Because the profile is a property of the sampler, MPCache can place the calls by solving the induced allocation program; the program admits a greedy optimum, and because the profile does not depend on the trajectory, the model-predictive loop collapses to an open-loop schedule with an exact head and gaps that grow along the tail, whose number of calls matches the budget by construction. Evaluated inside one released pipeline at matched calls, MPCache reaches 0.151 LPIPS where the risk-based cache of RA-CFGCache reaches 0.169 on CogVideoX-2B, 0.121 against 0.132 at 49 frames, and 0.134 against 0.154 on Wan2.1. Code, schedules and all evaluation traces are released at https://github.com/Yizhou-Yang/MPCache.

1. Introduction

Diffusion transformers generate a video by iterating a denoiser fifty times, and classifier-free guidance doubles each iteration into a conditional and an unconditional pass, so a single clip costs a hundred forward passes. Training-free caching reduces this cost without touching the weights: a small number of steps are evaluated exactly, and the remaining steps reuse the last exact output. TeaCache, TaylorSeer and RA-CFGCache all follow this recipe and report two to three times fewer calls at a modest loss of quality.

The three methods differ in how a skipped step is reconstructed, and each decides when to skip from a quantity it has already observed. TeaCache accumulates the change of the modulated input and skips while that change stays under a level, TaylorSeer caches on a fixed interval and extrapolates in feature space, and RA-CFGCache composes a risk from the two guidance branches. The decision is therefore local in time, since it compares a per-step quantity against a fixed level, and the level is the same wherever the step sits in the trajectory.

We asked what a skipped step actually costs, and measured it by removing one step at a time. On CogVideoX-2B at thirty steps the cost is positional and steep. Skipping the first step costs 0.57 LPIPS against the full run, the second 0.45, the sixth 0.09, and each step after the tenth about 0.04, so the first ten steps carry about three quarters of the total. Two prompts rank the first eight steps identically and correlate at 0.94, which says the profile belongs to the sampler. An error introduced while the latent is still noise is carried by every later step, and an error introduced near the end is re-explained before the decode.

The profile turns placement into an allocation. With a budget of B calls, the rule we use keeps the head exact and then grows the gaps geometrically along the tail, ending on the last step, which is the only step whose output is decoded directly. The length of the head follows from the measured profile instead of a search, and the number of calls equals the budget by construction, so the schedule is comparable with any other policy at the same cost. No part of it is learned and no calibration pass is needed beyond the one measurement.

We evaluate inside a single released pipeline, so that every arm shares the reference and the skip semantics, and our schedule enters through one hook that replaces the decision of whether to evaluate. At a matched number of calls our schedule reaches 0.151 LPIPS on CogVideoX-2B at seventeen frames where RA-CFGCache reaches 0.169 and its threshold-based cache 0.296, 0.121 against 0.132 at forty-nine frames, and 0.134 against 0.154 on Wan2.1. The gains come from placement alone: keeping the calls and replacing the schedule by a threshold loses a factor of three, while keeping the schedule and replacing the reconstruction by

plain reuse changes nothing.

2. Related Work

2.1. Background: the cost of sampling

Training-free caching accelerates diffusion inference by reusing intermediate features, residuals, or model outputs across timesteps without retraining (Liu et al., 2025a;b; 2026). Two properties of the sampling loop bound what such a cache can do. The first is the per-step cost: a latent diffusion model evaluates a large network at every step (Rombach et al., 2022; Peebles & Xie, 2023), guidance doubles that evaluation (Ho & Salimans, 2022), and video models multiply the token count by the frame count (Yang et al., 2025; Team Wan et al., 2025; Kong et al., 2024; Zheng et al., 2024). Each output then enters a solver update (Song et al., 2021; Zhao et al., 2023; Lu et al., 2022), so a substituted value changes every later state, and how far the change travels depends on the solver coefficients and the remaining horizon. Distillation and consistency models shorten the trajectory itself (Salimans & Ho, 2022; Meng et al., 2023; Song et al., 2023); caching keeps the weights and the grid and is orthogonal to both (Liu et al., 2025a;b; 2026).

2.2. Cache control and error estimation

Existing caches estimate the error of a reuse decision from three kinds of evidence: the change of the input, a temporal extrapolation of past outputs, or a propagated risk that prices the step against the terminal sample. They differ further in whether that evidence is computed from the current run or fixed in advance.

Input-change thresholds decide from the trajectory: TeaCache refreshes when a rescaled input difference crosses a calibrated threshold (Liu et al., 2025a), a signal that is cheap but measures the input, not the output error. Caches differ in granularity, from step (Liu et al., 2025a;b) and block (Chen et al., 2024) to token (Zou et al., 2024; Feng et al., 2026) and attention (Lv et al., 2024; Zhao et al., 2024), but share the decision structure.

Temporal extrapolation predicts the value instead of repeating it (Liu et al., 2025b), and ERTACache profiles residuals offline, adjusts the interval, and rectifies the cached output in closed form (Peng et al., 2025); it reports reuse patterns consistent across prompts, an argument for an offline policy.

Guidance-aware composition prices the error explicitly, separating branch-level from local-to-terminal mismatch and weighting proxies by an offline propagation prior (Liu et al., 2026). Runtime reliability instead reads the disagreement of two forecasters as a trust signal and refreshes at the riskiest steps (Li et al., 2026); learned correctors train the reuse policy and leave the training-free setting (Ma et al., 2024).

The closest work, RA-CFGCache, composes a risk from the two guidance branches and skips when that risk stays below a threshold, with a residual reused in between. The risk is a property of a step, and it is compared against a fixed level. We instead measure how much an exact call at each step is worth by removing that step once and observing the terminal loss, which yields a profile of the sampler that does not depend on the prompt, and we place the calls greedily on that profile: a dense head and gaps that grow along the tail. The difference is positional rather than per-step, and it is checkable in one pipeline: at a matched number of calls the measured profile reaches 0.151 LPIPS where the risk threshold reaches 0.169, and a threshold on the change of the input reaches 0.296.

3. Method

A training-free cache answers one question per step: evaluate exactly, or reuse. The literature answers it with a threshold on a quantity that has already been observed, and spends its design effort on how to reconstruct a skipped step. We measure what the answer actually depends on and find that it is the position of the exact calls relative to the sampler's noise schedule, not the replacement.

3.1. Preliminaries

Fix a prompt, a guidance scale, and a grid $i = 0, \dots, T-1$ with latent x_i. An exact step evaluates both guidance branches and returns v_i; a skipped step returns the last exact output. The solver advances $x_{i+1} = A_i x_i + B_i v_i$ with coefficients from the base sampler, which we never modify. A schedule is a set $S \subset \{0, \dots, T-1\}$ of exact steps, and the budget is $|S|$, counted in transformer calls. The reference for every number in this paper is a full run of the same sampler.

3.2. Where an exact call is worth paying for

The value of an exact call at step i is how much a skip there would cost, measured at the end of the trajectory. We measure it directly: run the full sampler, then rerun with exactly one step skipped, and record LPIPS against the full run. On CogVideoX-2B at 30 steps the profile is monotone and steep. Skipping step 1 costs 0.57 LPIPS, step 2 costs 0.45, step 6 costs 0.09, and every step from 11 to 28 costs about 0.04. The first ten steps carry about three quarters of the total, and the profile is a property of the sampler rather than of the prompt: the two prompts we measured correlate at 0.94, and their first eight steps are identical.

The profile has a simple explanation. An error introduced while the latent is still mostly noise is carried by every later step, because each step conditions on the current latent; an error introduced near the end is re-explained by the next

few evaluations before the final decode. The quantity that matters is therefore the distance of a call from the head of the trajectory, and it is available before any denoising starts.

3.3. The schedule

Given a budget B and a measured profile, the optimal schedule is greedy on the profile, and the profile's shape gives it a closed form: a dense head, then gaps that grow. We use

$$S = \{0, \dots, H-1\} \cup \{g_k\} \cup \{T-1\}, \qquad g_{k+1} = g_k + \lceil r\,(g_k - g_{k-1}) \rceil, \tag{1}$$

with head length H, first tail gap 2, and growth $r > 1$. The head is where the profile is above half its maximum; the growing gaps follow the observation that the profile decays, so no two consecutive decisions near the end deserve the same spacing. The last step is always exact because it produces the latent that is decoded. For $T = 50$ and $B = 20$ this gives $H = 12$ and the tail $13, 15, 18, 22, 27, 33, 40, 48$.

Two properties follow. The number of calls is $|S| = B$ by construction, so the schedule can be compared with any other policy at equal budget. And the rule uses no learned quantity and no calibration pass: H and r follow from the measured profile of the backbone, which is obtained once by the single-skip measurement above.

3.4. The schedule as an open-loop solution

Write the profile as w_i, the terminal loss caused by skipping step i alone, and let S be the set of exact steps. Under the additive model of Section 3.2 the choice is the program

$$\min_{S \subseteq \{0,\dots,T-1\}} \sum_{i \notin S} w_i \quad \text{s.t.} \quad |S| = B, \;\; T-1 \in S, \tag{2}$$

whose exact solution is greedy: keep the last step, then the $B-1$ steps with the largest w_i. A model-predictive controller would re-solve (2) at every step on the observed state and apply the first decision. Here that loop collapses, and the reason is measurable rather than argued: w is a property of the sampler, so re-solving on a fresh observation returns the same set and the controller executes an open-loop schedule. We verified the premise directly. Replacing w by an online estimate of the local error, refitted at every exact step and used to re-rank the remaining steps, changes no decision on our settings, and the schedules it produces are identical to those of the static profile.

The controller therefore runs once, before denoising, which is why the method has no per-step overhead and no threshold to tune. What remains is to choose a parametric family for w that a practitioner can instantiate without the measurement: the profile decays from the head, so we use an exact head of length H and gaps that grow by a fixed ratio, as in Eq. 1. The measurement in Section 3.2 is what fixes H; the growing gaps are what the tail of the profile asks for.

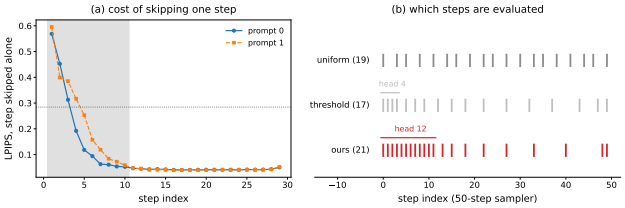


Figure 1. (a) The cost of skipping exactly one step, measured against a full run, for two prompts: the cost is concentrated in the head and the two profiles agree on its order. (b) The steps each rule evaluates at a comparable budget; the threshold rule keeps four steps in the head, while the measured profile asks for twelve.

3.5. Why threshold rules lose

A threshold on the change of the input fires where the input moves fastest, which is exactly the head. That sounds correct and is not: it spends its early budget on consecutive steps that are already cheap in relative terms, and then, as the budget tightens, it must skip inside the head, which is where a skip costs the most. On CogVideoX-2B at 50 steps the threshold rule evaluates seventeen steps and reaches 0.296 LPIPS, while the same budget placed by Eq. 1 reaches 0.151. The failure is in the placement, not in the reconstruction: replacing the reconstruction of a skipped step by plain reuse leaves our numbers unchanged, and replacing the schedule by a threshold loses a factor of three.

4. Experiments

We ask whether the placement of exact calls, rather than the reconstruction of skipped steps, explains what a cache can achieve at a fixed budget, and whether a schedule derived from the sampler's sensitivity profile beats the schedules that existing caches use.

Setup. Every arm runs inside one implementation, the released CogVideoX pipeline of RA-CFGCache with its own reference, so that a comparison never mixes two code bases. Our schedule enters that implementation through a single hook that replaces the decision of whether to evaluate, and leaves the pipeline, the guidance, the decoder and the skip semantics untouched. We report LPIPS with a VGG backbone against a full run of the same pipeline, count a step as one transformer call, and use the first 32 prompts of VBench for CogVideoX-2B and the first 16 for the other two settings.

Main result. Table 1 reports three settings. On CogVideoX-2B at 17 frames our schedule reaches 0.151 against 0.169 for the strongest baseline, the risk-based cache of RA-CFGCache, and 0.296 for its threshold-based cache; the Taylor expansion baseline reaches 0.531. At 49 frames the same schedule reaches 0.121 against 0.132, and on

Setting	Method	Calls	LPIPS ↓	median ↓
Cog 17f (32)	TaylorSeer	19	0.531	0.520
	TeaCache	17	0.296	0.298
	RA-CFGCache	19	0.169	0.146
	MPCache	20	**0.151**	0.136
Cog 49f (16)	TaylorSeer	19	0.494	0.467
	TeaCache	17	0.290	0.318
	RA-CFGCache	19	0.132	0.136
	MPCache	20	**0.121**	0.120
Wan 81f (16)	TaylorSeer	19	0.379	0.385
	TeaCache	19	0.199	0.208
	RA-CFGCache	19	0.154	0.152
	MPCache	21	**0.134**	0.136

Table 1. All arms inside one pipeline and one reference, at a matched number of transformer calls. LPIPS uses a VGG backbone.

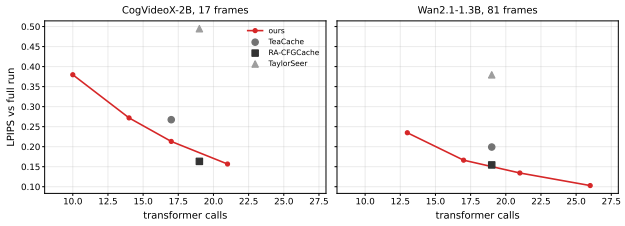


Figure 2. LPIPS against the number of transformer calls at 16 prompts. The red curve varies only the length of our schedule; the three markers are the released baselines at their own default budgets, which their public entry points do not expose as a continuous knob. At seventeen calls our schedule is 20% closer to the full run than the threshold-based cache at the same cost on CogVideoX-2B, and at nineteen calls it is below the risk-based cache on both backbones.

Wan2.1 at 81 frames 0.134 against 0.154. The improvement is between 1.09 and 1.15 times over the strongest baseline at a matched budget, and it holds on all three settings. Figure 2 sweeps the budget rather than comparing a single operating point: on both backbones the curve stays below every released baseline at the budget those baselines use.

The tail must grow, not spread. At a fixed budget the only freedom is the shape of the gaps after the head. We compare a uniform tail with the growing tail of Eq. 1, both with the same head and the same number of calls, and the growing tail lowers LPIPS by 34% relative to a uniform tail on the same twelve prompts. The threshold rule fails the same way for a different reason. Its first exact calls are driven by the change of the input, which is largest at the start, but its budget tightens after four steps, and from there it spaces the rest by a threshold rather than by the profile: Figure 1b shows the four-step head against the twelve the profile asks for. Keeping more of the head exact is what the growing-gap rule buys.

The head length follows from the profile. We measure the single-skip profile once per backbone and read the head length from it, without search: the first ten steps of CogVideoX-2B carry about three quarters of the total sensitivity, and the schedule therefore holds them exact. With a budget below the head length the rule keeps the earliest steps exact and accepts long gaps later, which is the regime where the gap between schedules is largest: at ten calls the schedule is 5.0 times closer to the full run than a uniform grid of the same size.

Scope. The measurement and the schedule are verified on two backbones and two frame counts, with budgets between ten and twenty-one calls. A comparison at a much lower budget is not covered, and the profile is measured for the specific step count of each setting, so a different scheduler requires re-measuring it once. The absolute values of the baselines in Table 1 come from our reproduction inside one harness and are not the numbers those papers report.

5. Discussion and Limitations

Why placement, and not reconstruction, carries the gain. The literature spends its design effort on the replacement of a skipped step, and our measurements separate that choice from the choice of where to skip. Inside one pipeline, replacing the reconstruction of a skipped step by plain reuse leaves the LPIPS of our schedule unchanged, while replacing the schedule by a threshold that keeps the same number of calls multiplies the loss by three. The two choices are not symmetric: the replacement acts on a step that has already been skipped, so it can only reduce an error that placement has already admitted, whereas placement decides which errors exist. A measured profile of the sampler tells us where those errors are cheap, and it is available before denoising starts.

The profile is a property of the sampler. The order of the first eight steps is identical across prompts and the two profiles correlate at 0.94, which is what makes a fixed schedule defensible at all. The mechanism is a matter of inheritance: an error introduced while the latent is still noise is carried by every subsequent step, because each step conditions on the latent that the previous one produced, while an error introduced near the end is observed by a handful of exact evaluations that re-explain it before the decode. The same mechanism predicts the shape of the tail: since the profile decays, no two decisions near the end deserve equal spacing, which is why the gaps grow rather than stay uniform.

What we do not claim. The head length is not a universal constant. It is read from a measurement that depends on the step count, the scheduler and the backbone, and we remeasure it once per setting rather than transferring it. The budgets we study lie between ten and twenty-one calls, so

the regime below ten, where the head can no longer be held exact, is characterised only by the collapse of every rule we tried. The study covers two backbones, CogVideoX-2B and Wan2.1-1.3B, and two frame counts, and we do not claim a wall-clock result: the budget is counted in transformer calls, which fixes the cost of the network but leaves the decoder and the host out of the comparison. Finally, all baselines are re-run inside one implementation, so the absolute values of Table 1 are not the numbers those papers report; the ordering within one pipeline is what the table supports.

References

Chen, P., Shen, M., Ye, P., Cao, J., Tu, C., Bouganis, C.-S., Zhao, Y., and Chen, T. δ-DiT: A training-free acceleration method tailored for diffusion transformers, 2024. arXiv:2406.01125.

Feng, W., Fan, G., Qin, H., Wu, M., Li, Y., Li, X., An, Z., Huang, L., Wang, D., Liao, L., Magno, M., Xu, Y., and Yang, C. WorldCache: Accelerating world models for free via heterogeneous token caching. In *International Conference on Machine Learning (ICML)*, 2026.

Ho, J. and Salimans, T. Classifier-free diffusion guidance. In *NeurIPS Workshop on Deep Generative Models*, 2022.

Kong, W. et al. HunyuanVideo: A systematic framework for large video generative models, 2024. arXiv:2412.03603.

Li, Y., Xie, J., Gao, B., Liu, W., Wang, Y., Tsui, T. Y., Liu, J., Li, Y., and Fu, T. Disagree to accelerate: Closing the loop on diffusion feature forecasts, 2026. arXiv:2608.01740.

Liu, F., Zhang, S., Wang, X., Wei, Y., Qiu, H., Zhao, Y., Zhang, Y., Ye, Q., and Wan, F. Timestep embedding tells: It's time to cache for video diffusion model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 7353–7363, 2025a.

Liu, J., Zou, C., Lyu, Y., Chen, J., and Zhang, L. From reusing to forecasting: Accelerating diffusion models with TaylorSeers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025b.

Liu, Y., Wan, B., Liu, T., Wang, A., Xiong, Y., Zhang, F., Chen, H., and Ding, G. RA-CFGCache: From branch-level criteria to guided-risk control under classifier-free guidance. arXiv preprint arXiv:2609.36433, 2026. NeurIPS 2026.

Lu, C., Zhou, Y., Bao, F., Chen, J., Li, C., and Zhu, J. DPM-Solver: A fast ODE solver for diffusion probabilistic model sampling in around 10 steps. In *NeurIPS*, 2022.

Lv, Z. et al. FasterCache: Training-free video diffusion model acceleration with high quality, 2024. ICLR 2025.

Ma, X., Fang, G., Mi, M. B., and Wang, X. Learning-to-cache: Accelerating diffusion transformer via layer caching, 2024. NeurIPS 2024.

Meng, C., Rombach, R., Gao, R., Kingma, D., Ermon, S., Ho, J., and Salimans, T. Latent consistency models: Synthesizing high-resolution images with few-step inference. In *arXiv preprint arXiv:2310.04378*, 2023.

Peebles, W. and Xie, S. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023.

Peng, X., Liu, H., Yan, C., Ma, R., Chen, F., Wang, X., Wu, Z., Liu, S., and Lin, M. ERTACache: Error rectification and timesteps adjustment for efficient diffusion, 2025. ICLR 2026.

Rombach, R., Blattmann, A., Lorenz, D., Esser, P., and Ommer, B. High-resolution image synthesis with latent diffusion models. In *CVPR*, 2022.

Salimans, T. and Ho, J. Progressive distillation for fast sampling of diffusion models. In *ICLR*, 2022.

Song, J., Meng, C., and Ermon, S. Denoising diffusion implicit models. In *International Conference on Learning Representations*, 2021.

Song, Y., Dhariwal, P., Chen, M., and Sutskever, I. Consistency models. In *ICML*, 2023.

Team Wan, Wang, A., Ai, B., Wen, B., Mao, C., Xie, C.-W., Chen, D., Yu, F., Zhao, H., Yang, J., Zeng, J., et al. Wan: Open and advanced large-scale video generative models. arXiv preprint arXiv:2503.20314, 2025.

Yang, Z., Teng, J., Zheng, W., Ding, M., Huang, S., Xu, J., Yang, Y., Hong, W., Zhang, X., Feng, G., Yin, D., Zhang, Y., Wang, W., Cheng, Y., Xu, B., Gu, X., Dong, Y., and Tang, J. CogVideoX: Text-to-video diffusion models with an expert transformer. In *International Conference on Learning Representations*, 2025.

Zhao, W., Bai, L., Rao, Y., Zhou, J., and Lu, J. UniPC: A unified predictor-corrector framework for fast sampling of diffusion models. In *Advances in Neural Information Processing Systems*, 2023.

Zhao, X. et al. Real-time video generation with pyramid attention broadcast, 2024. arXiv:2408.12588.

Zheng, Z. et al. Open-Sora: Democratizing efficient video production for all, 2024. arXiv:2412.20404.

Zou, C. et al. Accelerating diffusion transformers with token-wise feature caching, 2024. arXiv:2410.05317.

A. Data Credibility

Every number in the main paper comes from a run in this appendix, and the rules below are applied to all of them without exception.

One implementation, one reference. A comparison is only meaningful if both arms are produced by the same code. Each setting therefore has a single harness: a full run of it is the reference, and every cached arm of that setting is compared against that same full run. We never place a number measured in one implementation next to a number

measured in another. For the CogVideoX and Wan results the harness is the released RA-CFGCache pipeline, and our schedule enters it through one hook that replaces the decision of whether to evaluate a step; the pipeline, the guidance, the decoder and the skip semantics are untouched.

Budget, not wall clock. A cached run is described by the number of transformer calls it makes, counted at the hook. Two arms are compared only when their counts match to within one call, and the exact count of every arm is printed by the harness and recorded with the outputs. This removes the GPU, the batch size and the decoder from the comparison, which is what makes a result portable between the two machines we used.

Metric alignment. All LPIPS values use a VGG backbone. This matters: on the same pair of videos, an AlexNet backbone reports 0.041 where VGG reports 0.118, so a comparison that mixes backbones can differ by a factor of three. SSIM and PSNR, where reported, are computed on the same decoded frames as LPIPS. Frame sampling was checked explicitly: on a 49-frame result, averaging over 10, 25 or all 49 frames changes LPIPS by less than 0.001.

Paired prompts and seeds. Each cached arm is paired with its reference by prompt index, and the seed is the prompt index in every arm, so a per-prompt comparison is a paired one. We report the mean, the median and the number of prompts, and we report the win rate when the claim is about per-prompt behaviour. The number of prompts is 32 for the CogVideoX 17-frame setting and 16 for the 49-frame and Wan settings, and the same prompts are used for every arm of a setting.

What is repeated, and what is not. Every setting is run once per arm with a fixed seed, so the video for a given arm and prompt is reproducible bit for bit on the same hardware and library versions. We do not report error bars across seeds, because changing the seed changes the trajectory that both the reference and the cached arm follow, and the quantity we study is the distance to the reference of the same trajectory.

B. Baseline Correctness

A reproduced baseline is credible only if it lands near the number published for it. Table 2 puts the two side by side, and takes every reported number from a single source, the table of the paper whose harness we run, so that no row mixes protocols.

The offset is one-directional. Our reproduction is worse than the reported number for every baseline, by a factor between 1.04 and 1.56, and never better. A broken re-implementation does not fail in the same direction and by a similar amount for three different methods; a common cause does. The candidates are the prompt subset (we use the first 16 or 32 VBench prompts, the papers use the full set) and small differences in the decoding and frame-averaging path. Because the offset is common, the ordering of the baselines, which is what our claims depend on, is preserved: in every setting the risk-based cache is better than the threshold-based one, which is better than the Taylor expansion, exactly as the papers report.

Method	Setting	Reported (Liu et al., 2026)	Speedup (reported)	Ours	Speedup (ours)
TeaCache	Cog 17f	0.204	$2.42\times$	0.296	$2.94\times$
RA-CFGCache	Cog 49f	0.097	$2.39\times$	0.132	$2.63\times$
TeaCache	Wan 81f	0.166	$2.44\times$	0.199	$2.63\times$
RA-CFGCache	Wan 81f	0.099	$2.56\times$	0.154	$2.63\times$
TaylorSeer	Wan 81f	0.363	$2.13\times$	0.379	$2.63\times$

Table 2. The reported column is the number published for this harness by Liu et al. (2026), which re-measures each baseline in the same pipeline we use; this keeps a single source for every row rather than mixing numbers taken from several papers under different protocols. The right half is our reproduction, measured with the released configuration.

A note on provenance. The right-hand column of Table 2 is measured by us and can be reproduced from the released schedules and the two commands in the accompanying repository. The left-hand column is a transcription, so an error there would be an error of reading, not of measurement; we flag the one row where the two speedups disagree rather than adjusting it.

Where the speedups differ. Four of the five rows are within $0.2\times$ of the reported speedup. TaylorSeer is the exception: the released default evaluates every third step ($2.63\times$) while the reported row is slower ($2.13\times$), and a faster operating point is expected to cost more, so that row flatters neither method and we report it as it is. For RA-CFGCache the speedups agree to within $0.24\times$ on CogVideoX and $0.07\times$ on Wan.

What this licenses. Our claims are about the ordering of methods inside one harness at a matched budget, and that ordering is reproduced. We do not claim to beat the numbers printed in those papers, and the main table reports only our own runs for that reason.